

AURUM PHARMA

Aurum Pharma — onboarding guide для нового программиста

Дата обзора: 8 июля 2026

Репозиторий: Aurum-Pharma

Цель документа: быстро ввести нового разработчика в продукт, архитектуру, уже реализованные части и ближайшие задачи до релиза.

1. Что это за продукт

Aurum Pharma — SaaS-система автоматизации аптек для Республики Таджикистан. По смыслу это аналог 1С для аптек: касса, склад, партии и сроки годности, приход, поставщики, роли, отчёты, аудит, подписка и базовый onboarding.

Пилотная версия ориентирована на русский язык и валюту TJS. Продукт должен работать локально в разработке через Docker, затем как web-версия в браузере и как Windows-приложение на базе той же web-версии.

Главная идея: один общий backend и один frontend. Windows-приложение позже будет WebView2-оболочкой, которая добавит только интеграции с локальными устройствами: сканер, принтер чеков, денежный ящик и сохранение файлов.

2. Текущий статус

Статус проекта: MVP сильно продвинут. Основные backend-домены, frontend-экраны, миграции, RLS-защита, Docker-окружение, unit/integration/e2e-тесты уже есть.

Ключевые числа из текущего кода:

Область	Сейчас
Backend-домены	13
Frontend-фичи	15
Alembic-миграции	24
ORM-модели	34
Frontend routes	20
Docker-сервисы	9
Playwright E2E specs	11
Playwright E2E tests	28
Frontend Vitest tests	226 passed в последнем полном прогоне; статический grep сейчас находит 227 it/test
Последний полный backend pytest-прогон в рабочей сессии	249 passed

Важно: документация полезна, но источник истины — реальный код. Если документ и код расходятся, сначала проверяй код через rg.

3. Как запустить локально

Из корня проекта:

```
cd C:\Users\Asadov\Desktop\A-Pharma
docker compose up -d
docker compose exec backend alembic upgrade head
docker compose exec backend python -m app.seed_demo
powershell -ExecutionPolicy Bypass -File .\scripts\demo-smoke.ps1
```

Открыть:

- Frontend: <http://localhost:5173>
- Backend docs: <http://localhost:8000/docs>
- Healthcheck: <http://localhost:8000/healthz>
- MinIO: <http://localhost:9001>
- Prometheus: <http://localhost:9090>

Локальный launcher для Windows:

```
Start-Aurum-Pharma-Admin.cmd
```

Он поднимает Docker, применяет миграции, запускает demo-seed и smoke-проверку.

4. Архитектура простыми словами

Поток запроса:

```
React UI
-> Axios API client
-> FastAPI router
-> Service layer: бизнес-логика
-> Repository layer: SQLAlchemy-запросы
-> PostgreSQL с RLS
-> Audit triggers / Redis / Celery / MinIO при необходимости
```

Основные компоненты:

Компонент	Назначение
frontend/	React 18, TypeScript, Vite, TanStack Router/Query
backend/	FastAPI, SQLAlchemy async, Alembic, Celery
PostgreSQL 16	Основная БД, RLS, audit triggers
Redis	Кэш permissions, очереди Celery
MinIO	Файлы: импорты, будущие документы/экспорты
Celery worker/beat	Фоновые задачи и расписания

Playwright	Сквозные браузерные проверки
------------	------------------------------

Главный архитектурный принцип backend: каждый домен живёт в `backend/app/domains/<domain>/` и делится на `models.py`, `schemas.py`, `repository.py`, `service.py`, `router.py`.

Главный архитектурный принцип frontend: каждая фича живёт в `frontend/src/features/<domain>/` и делится на `api.ts`, `queries.ts`, `types.ts`, страницы и компоненты.

5. Backend: что уже сделано

Домен	Что реализовано
auth	Пользователи, email-коды, verify, refresh, logout, /me, rate-limit и login attempts
foundation	Аптеки/тенанты, настройки, филиалы, кассы, owner creation
roles	Permissions, роли, шаблоны, назначения, приглашения, anti-escalation
catalog	Каталог товаров, штрихкоды, поиск, CSV/XLSX импорт с preview/confirm/rollback
inventory	Партии, движения партий, списания, FEFO-подбор партий
suppliers	Поставщики и возвраты поставщикам
incoming	Приходные документы, позиции, принятие прихода в партии
pos	Смены, продажи, оплаты, рецепты, чеки, возвраты, Z-report, XLSX/PDF exports
billing	Тарифы, подписки, счета, платежи, trial/grace/readonly lifecycle
audit	Audit log через PostgreSQL-триггеры, поиск и redaction sensitive-полей
onboarding	Wizard, checklist, старт trial при готовности
notifications	In-app уведомления, подписки, очередь доставок, stubs для внешних каналов
dashboard	Сводка: продажи, смены, сроки, лицензии, счета, кэш Redis

Backend entrypoint: `backend/app/main.py`.

Подключение роутеров идёт там же через `app.include_router(...)`.

6. База данных и безопасность

Проект мульти-тенантный: одна БД обслуживает много аптек. Каждая tenant-таблица имеет `tenant_id`.

Критичная защита: RLS (Row Level Security). Это защита строк на уровне PostgreSQL. Обычный пользователь не должен увидеть строки другой аптеки даже при ошибке в Python-коде.

Как это работает:

- `current_tenant_id()` читает текущий tenant из PostgreSQL GUC.
- `is_support_session()` разрешает обход только support-сессиям.
- `aurum_app` — обычная роль БД, RLS включён.
- `aurum_support` — роль для developer/administrator, имеет BYPASSRLS.
- Middleware читает JWT и выставляет контекст запроса.

Важно для нового разработчика:

- не обходить RLS Python-флагами;
- не запускать alembic downgrade на общей dev-БД;
- не делать docker compose down -v, если не хочешь удалить локальную БД;
- не хранить permissions большим массивом в JWT;
- не логировать PII: email, телефоны, данные пациентов, содержимое чеков, закупочные цены.

7. Frontend: что уже сделано

Основные routes:

Route	Экран
/	Dashboard
/login	Вход
/admin/tenants	Управление аптеками для dev/admin
/branches	Филиалы
/registers	Кассы
/settings	Настройки аптеки
/users	Пользователи
/roles	Роли
/catalog	Каталог товаров
/batches	Партии и остатки
/suppliers	Поставщики
/incoming	Приходные документы
/incoming/\$id	Детали прихода
/pos	Касса
/sales	Продажи и возвраты
/billing	Подписка, счета, платежи
/audit	Аудит
/onboarding	Onboarding checklist

/notifications	Уведомления
/reports	Отчёты

Уже есть:

- общий AppLayout, sidebar, protected routes;
- UI primitives в frontend/src/components/ui;
- TanStack Query hooks для серверных данных;
- формы через React Hook Form + Zod;
- POS с поиском товара, сканером, draft restore, оплатами, печатью чека, desktop bridge;
- PWA manifest/service worker без кеширования API/HTML;
- offline/server status banners;
- runtime detection: browser, pwa, windows-desktop.
- светлая/тёмная/системная тема уже реализована, хотя в старой спецификации тёмная тема была отнесена к Этапу 2.

8. POS и склад

POS — самая критичная часть продукта.

Уже реализовано:

- открытие/закрытие смены;
- создание draft-продажи;
- добавление товара по поиску или штрихкоду;
- оплата наличными, картой, переводом;
- завершение продажи;
- запрет изменения completed-продажи;
- возвраты через отдельную sale-строку;
- FEFO: списание сначала партий с ближайшим сроком годности;
- чек в UI и PDF;
- Z-report;
- экспорт отчётов XLSX;
- desktop barcode event;
- desktop cash drawer event после успешной наличной продажи.

Правило для нового разработчика: completed sale нельзя обновлять. Исправление или отмена делается новой строкой возврата.

9. PWA и Windows-приложение

Web-версия уже подготовлена как installable PWA:

- manifest.webmanifest;

- service worker только для статических frontend-файлов;
- API, токены, чеки и цены не кешируются;
- warning при offline-режиме.

Windows-направление уже подготовлено документами и контрактом:

- docs/desktop-bridge.md;
- docs/desktop-host-implementation.md;
- scripts/windows-host-readiness.ps1;
- scripts/windows-host-setup.ps1;
- scripts/windows-host-scaffold.ps1.

План Windows-приложения: WinUI 3 + WebView2 host. Frontend остаётся тем же. Desktop host добавляет bridge:

```
window.aurumDesktop = {
  appVersion: "0.1.0",
  platform: "windows",
  capabilities: ["receipt-print", "barcode-scanner", "cash-drawer", "file-export"],
  postMessage(message) {
    window.chrome.webview.postMessage(message);
  },
};
```

Что уже поддерживает frontend:

- aurum.desktop.ready;
- aurum.receipt.print;
- aurum.cash-drawer.open;
- aurum.file-export.request;
- aurum-desktop-barcode-scanned.

Что ещё не сделано: реальный WinUI host, работа с физическим оборудованием и подключение native receipt-print к UI. Сейчас чек можно печатать web-flow/PDF, а bridge-контракт для нативной печати уже описан.

10. Тесты и quality gates

Backend:

```
docker compose exec backend pytest tests/ -v
docker compose exec backend ruff check app tests
docker compose exec backend black --check app tests
docker compose exec backend mypy app
```

Frontend:

```
docker compose exec frontend pnpm typecheck
docker compose exec frontend pnpm lint
```

```
docker compose exec frontend pnpm test  
docker compose exec frontend pnpm build
```

E2E:

```
docker compose restart frontend  
cd frontend  
pnpm e2e
```

Что покрыто E2E:

- login/logout;
- tenant setup;
- owner onboarding;
- catalog CRUD/search/barcode/import;
- incoming draft -> accepted -> batch;
- POS sale с FEFO split;
- shift close -> Z-report;
- report exports;
- PWA/runtime/desktop bridge.

Риск: CI сейчас не запускает весь frontend test/build и Playwright E2E. Это надо усилить до релиза.

Ещё один фронтенд-риск: отчёт по списаниям в ReportsPage ожидает отдельный backend endpoint. Это не блокирует текущие продажи, но важно для полного отчётного контура.

11. Что уже готово для MVP

С точки зрения продукта:

- базовый multi-tenant SaaS;
- локальный Docker dev-stack;
- вход по email-коду в dev-режиме;
- управление аптеками, филиалами и кассами;
- роли, permissions и пользователи;
- каталог ЛС и штрихкоды;
- импорт каталога CSV/XLSX;
- партии и сроки годности;
- приход и поставщики;
- касса и продажи;
- чеки, возвраты, Z-report и отчёты;
- billing lifecycle;
- audit log;
- onboarding;
- уведомления;
- PWA-основа;

- desktop bridge contract;
- широкий набор тестов.

Это уже не “пустой scaffold”, а рабочий MVP-каркас.

12. Что нужно сделать до полного релиза

Обязательное перед пилотом:

1. Прогнать полный checklist на чистой/staging среде.
2. Усилить CI: frontend unit tests, frontend build, Playwright E2E, миграции.
3. Подключить реальную SMTP-доставку login/invite кодов.
4. Настроить production secrets: JWT, DB, MinIO, Redis, CORS, TLS.
5. Настроить backup/restore PostgreSQL и MinIO.
6. Провести security review RLS, JWT, CORS, storage tokens.
7. Проверить POS на параллельные продажи и конкуренцию за партии.
8. Подготовить staging/prod deploy-артефакты.
9. Проверить реальное оборудование: сканер, чековый принтер, денежный ящик.
10. Оформить минимальные инструкции для пилотной аптеки.

Технический долг/риски:

- refresh-токен пока хранится в localStorage; httpOnly cookie запланирован на Этап 2;
- notification_subscription и notification_delivery стоит отдельно проверить на tenant-защиту;
- внешние notification/email каналы пока stub;
- offline-касса не реализована;
- production hosting ещё не выбран;
- формальный SLA, WAF/IDS/pentest — позже.
- есть drift документации: docs/handoff.md описывает план на первые 12 миграций, а в коде уже 24 миграции и дополнительные UI-разделы; docs/спес-v3.md местами отстаёт от факта, например по тёмной теме.

13. Этап 2

После MVP и первых пилотов:

- httpOnly cookie для refresh-токенов;
- offline POS через очередь продаж;
- импорт из 1С:Аптека;
- эквайринг карт;
- PDF-экспорт всех отчётов;
- интерактивные dashboard drill-down;
- контроль цен и наценок;
- recall партий;

- реальная multi-currency логика;
- тёмная тема;
- 2FA для всех пользователей;
- расширенная безопасность;
- встроенный чат поддержки;
- staging/prod observability.

14. Этап 3

Более поздний релиз:

- фискализация РТ и ОФД;
- маркировка ЛС;
- учёт наркотических ЛС;
- таджикский и английский языки;
- импорт из локальных ERP РТ;
- программа лояльности;
- мульти-сеть аптек;
- полноценное desktop-приложение с оборудованием;
- ML-прогнозы закупок и сроков годности;
- bug bounty.

15. Как новичку безопасно начать работу

Рекомендованный первый день:

1. Прочитать AGENTS.md / CLAUDE.md.
2. Прочитать этот onboarding guide.
3. Запустить проект локально через Docker.
4. Открыть /login, войти как owner@aurum.tj.
5. Пройти /catalog, /incoming, /batches, /pos, /sales, /reports.
6. Посмотреть backend/app/main.py и frontend/src/router.tsx.
7. Запустить smoke:

```
powershell -ExecutionPolicy Bypass -File .\scripts\demo-smoke.ps1
```

Первая неделя:

- день 1: запуск, обзор UI и API docs;
- день 2: backend домены auth, foundation, roles;
- день 3: catalog, inventory, incoming, suppliers;
- день 4: pos, sales, reports;
- день 5: тесты, e2e, desktop bridge, CI gaps.

Правила работы:

- сначала читать код, потом менять;
- не делать попутный рефакторинг;
- не вайпать dev-БД;
- не запускать alembic downgrade на общей dev-БД;
- любые новые домены делать с тестами;
- после изменений прогонять релевантный checklist;
- коммиты делать маленькими и понятными.

16. Где что искать

Что нужно	Где смотреть
Общие правила проекта	AGENTS.md, CLAUDE.md
Функциональная спецификация	docs/сpec-v3.md
Схема БД	docs/db-schema-v2.md
План по доменам	docs/handoff.md
Локальный запуск	README.md, docs/local-demo.md
E2E	docs/e2e.md, frontend/e2e
PWA	docs/pwa.md
Desktop bridge	docs/desktop-bridge.md
Windows host plan	docs/desktop-host-implementation.md
Backend endpoint	backend/app/main.py
Frontend routes	frontend/src/router.tsx
Docker stack	docker-compose.yml
CI	.github/workflows/ci.yml

17. Глоссарий

Термин	Простое объяснение
Tenant	Отдельная аптека или сеть аптек внутри общей SaaS-БД
RLS	Защита строк в PostgreSQL, чтобы аптеки не видели чужие данные
GUC	Переменная контекста PostgreSQL, куда middleware кладёт tenant/user
FEFO	First Expired, First Out: сначала продаём партии с ближайшим сроком
POS	Point of Sale: кассовый экран и логика продаж

Shift	Кассовая смена
Sale	Продажа; после completed считается неизменяемой
Audit log	Журнал изменений данных
Celery	Фоновые задачи и расписания
MinIO	S3-подобное локальное хранилище файлов
PWA	Устанавливаемое web-приложение
WebView2	Встроенный браузер Edge внутри Windows-приложения

18. Главная мысль для нового коллеги

Проект уже имеет рабочую основу и строгие правила. Не надо начинать заново и не надо менять стек. Лучший вклад нового программиста — аккуратно усиливать готовый MVP: закрывать релизные риски, улучшать CI, подключать production-инфраструктуру, доводить Windows-host и проверять реальные аптечные сценарии.